Usine logicielle LdP — préparation du PI

Oct 15, 2026 · @Antoin

Synthèse

L'usine logicielle se structure en 2 Fonctions de service techniques, 14 Features et 87 éléments : 52 Enablers, 24 User Stories et 11 Spikes. Le PI-1 en porte 61, le PI-2 26.

Le PI-1 livre une chaîne de bout en bout sur un périmètre pilote (1 microservice back, 1 front) : MR contrôlée, déploiement GitOps en dev et intégration sur SKE, bundle standalone installable, version produit v0.1. Le PI-2 généralise à tous les composants, ouvre la qualification et la production, étend les blocages et produit le dossier de livraison.

Trois principes structurent l'ensemble :

- **Construire une fois, livrer deux fois** : mêmes images, un seul manifeste d'assemblage par version produit, deux générateurs (Argo CD et standalone).
- **Règles portées par le groupe** : jobs obligatoires et approbations imposés par des politiques GitLab, bloquants d'abord sur le code nouveau.
- **Hors ligne par conception** : outils, plugins et bases de vulnérabilités passent par le sas ; c'est le premier chemin critique.

Six décisions sont à prendre en début de PI, chacune portée par un Spike :

Décision	Spike	Sprint
Édition SonarQube : l'analyse des MR exige au moins l'édition Developer	Q3.1	PI1-S1
Runners et build d'images sans privilège	D1.3	PI1-S1
Architecture GitOps : dépôts, promotion, rendu des manifestes	D4.1	PI1-S1
Modèle d'assemblage et variabilité par programme	D3.1	PI1-S1 à S2
Format standalone : Compose, Quadlet ou `podman kube play`	D6.1	PI1-S1 à S2
Secrets et configuration pour les deux cibles	D3.2	PI1-S2

Quatre actions conditionnent le démarrage : lancer le lot d'import par le sas, qualifier la licence SonarQube, obtenir l'accès au cluster SKE, désigner le pilote (voir Préparation du PI-1).

Analyse du besoin

Le besoin n'est pas un outil de plus : c'est une chaîne unique qui garantit que la version testée est exactement celle livrée, quels que soient le programme et la cible.

Contexte et contraintes

Élément de contexte	Conséquence pour l'usine
Produit composé de plusieurs microservices back et front, portés par plusieurs équipes	Une chaîne standard, maintenue par une équipe usine, que chaque dépôt inclut sans la recopier
Ligne de produits : socle commun, parties variables gérées à l'assemblage	Un objet « version produit » distinct des versions de composants ; plusieurs versions maintenues en parallèle
Cible Kubernetes SKE gérée en GitOps par Argo CD	L'état de chaque environnement est décrit dans Git ; la CI ne déploie pas, elle propose une version

Élément de contexte	Conséquence pour l'usine
Cible standalone sans accès au cluster, sous docker ou podman	Un paquet hors ligne autoportant, généré depuis la même description que Kubernetes
Enclave hors ligne, proxy vers quelques dépôts autorisés	Miroirs internes pour images et paquets ; outils, plugins et bases importés par le sas
Pas de service de mail	Notifications par GitLab (to-do, statuts) ou webhooks ; aucun circuit ne dépend du mail
GitLab Ultimate 19+, administration de l'instance en main	Politiques de groupe, cadres de conformité, trains de merge, environnements protégés utilisables
2 PI de 4 sprints d'un mois	Un pilote de bout en bout au PI-1, la généralisation au PI-2

Des besoins aux exigences techniques

Besoin	Exigence technique	Features
Livrer la même version à plusieurs programmes, avec leurs variantes	Version produit décrite une fois, profils par programme, deux cibles générées	D3, D4, D6
Prouver la qualité de chaque version	Mesures homogènes, seuils opposables, rapport joint à la release	Q2, Q3, Q4, Q6
Vérifier les chaînes fonctionnelles sur le produit assemblé	Tests d'acceptation automatisés après chaque déploiement	Q5
Maîtriser les changements	Revue obligatoire, `main` toujours livrable, règles non contournables	Q1, Q6
Déployer vite et revenir en arrière	Déploiement GitOps automatique en dev, promotion par MR, revert	D4, D5
Installer sans cluster ni réseau	Bundle hors ligne, intégrité vérifiable, installation en une commande	D6, D7
Savoir ce qui est livré, où et pour qui	Traçabilité US → MR → version → programme, SBOM, signatures	D3, D7

Besoin	Exigence technique	Features
Garder des livrables sûrs	Secrets maîtrisés, vulnérabilités détectées et tracées	Q7, D3

Principes structurants

1. **Construire une fois, promouvoir partout** : l'image est construite au merge, puis promue par son empreinte (digest), jamais reconstruite.
2. **Code et déploiement séparés** : les dépôts applicatifs publient ; les dépôts d'assemblage et GitOps décrivent ce qui tourne, où.
3. **La chaîne est un produit** : composants CI versionnés au catalogue, modèles de projet, montées de version annoncées.
4. **Les règles vivent au niveau du groupe** : politiques d'exécution de pipeline et d'approbation, cadre de conformité, plutôt que des réglages dépôt par dépôt.
5. **Exigence progressive** : chaque contrôle passe par un mode observation, puis bloquant sur le code nouveau.
6. **Une seule source pour les deux cibles** : seuls l'orchestrateur et le point d'entrée diffèrent entre Kubernetes et standalone.

Hypothèses à confirmer avant le PI

- Langages back et front réellement utilisés : fixés par l'inventaire Q2.1.
- Moteur des cibles standalone : docker, podman ou les deux ; OS et réseau des sites.
- Argo CD fourni par l'équipe qui exploite SKE, ou installé par l'usine.
- Découpage du cluster : namespaces par environnement et par programme, ou cluster distinct pour la production.
- Coffre de secrets disponible dans l'enclave, ou chiffrement dans Git.
- Édition SonarQube déjà détenue ou à acquérir.

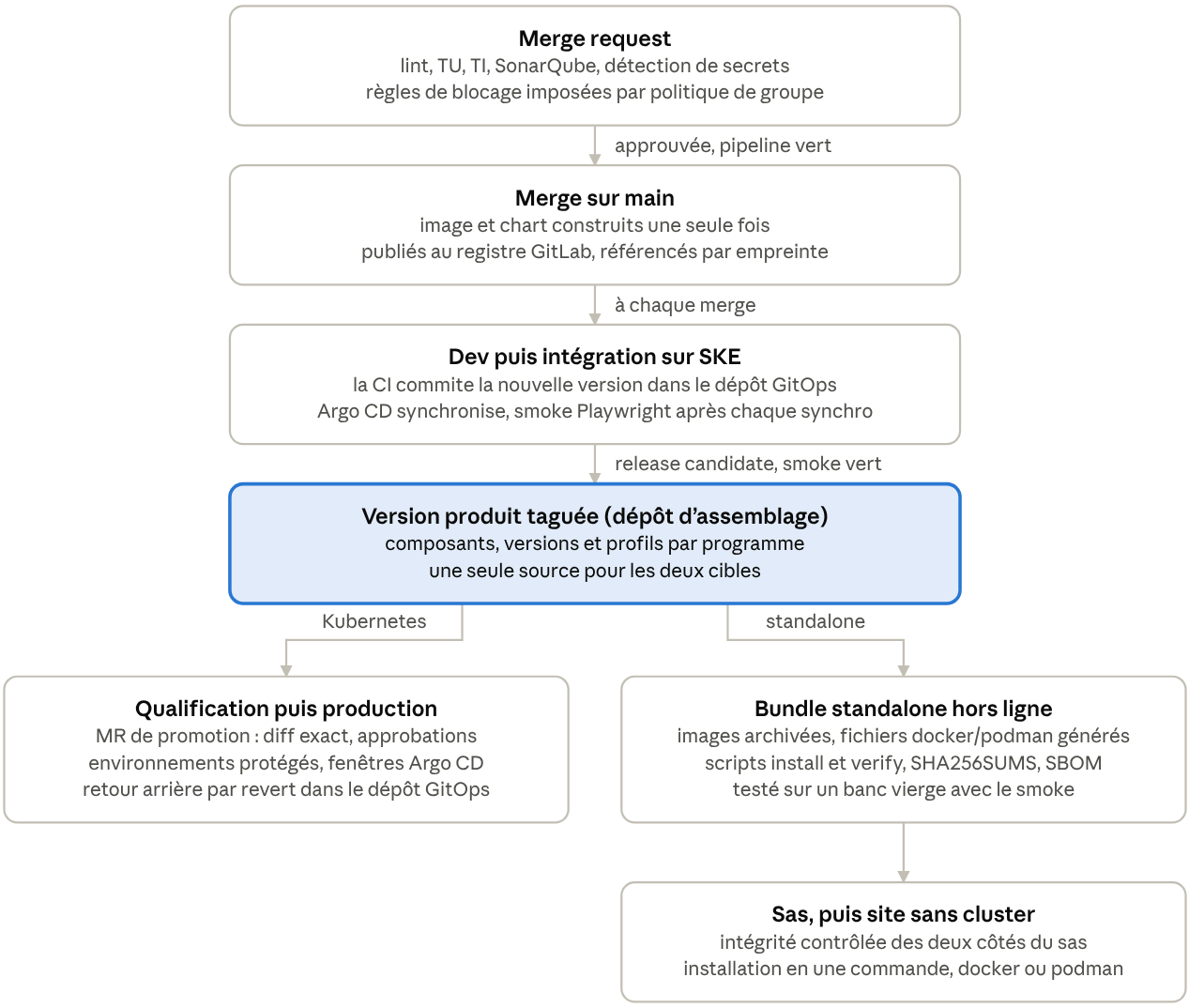
Hors périmètre proposé, à arbitrer : observabilité (journaux, métriques), tests de charge, plan de reprise de l'usine.

Architecture cible

L'usine tient en trois zones GitLab (usine, produit, livraison), un SonarQube, un Argo CD et des runners ; tout le reste est de la configuration versionnée.

Une version construite une fois, livrée sur deux cibles

Chaîne de bout en bout de la ligne de produits, de la merge request au site



chaîne de bout en bout · 7 étapes, 2 cibles

Le tag de version produit est le pivot : la même version part en qualification et en production par Argo CD, et dans un bundle hors ligne pour les sites sans cluster.

Organisation GitLab

Le code est rangé par architecture, qui est stable ; les backlogs restent rangés par équipe sous `ldp/10-equipes`, et les MR y font référence.

Chemin	Contenu	Responsable
`ldp/20-usine/ci-catalogue`	Composants CI/CD versionnés : lint, tests, sonar, build, gitops, bundle	Équipe usine
`ldp/20-usine/images`	Images de build et d'exécution par stack	Équipe usine
`ldp/20-usine/charts`	Chart Helm générique « microservice »	Équipe usine
`ldp/20-usine/politiques`	Projet de politiques de sécurité : exécution de pipeline, approbations	Équipe usine, sécurité
`ldp/20-usine/modeles-projets/`	Modèles de projet back et front, déclarés comme modèles du groupe	Équipe usine
`ldp/30-produit/socle/`	Microservices communs, un dépôt par service	Équipes produit
`ldp/30-produit/variantes/`	Composants propres à un programme	Équipes produit
`ldp/30-produit/tests-acceptation`	Suites Playwright du produit assemblé	Équipes produit, usine
`ldp/40-livraison/assemblage`	Manifeste de version produit, profils par programme	Responsable d'assemblage
`ldp/40-livraison/gitops`	État désiré de chaque environnement, lu par Argo CD	Usine (dev, intégration), release manager (qualification, production)
`ldp/40-livraison/standalone`	Générateur de bundle, scripts d'installation	Équipe usine
`ldp/modeles` (existant)	Modèles de description, dont les modèles de MR	Équipe usine

Éléments nécessaires

Élément	Rôle	Point d'attention en enclave
Runners GitLab	Exécuter les jobs, construire les images sans privilège (Buildah ou BuildKit rootless)	Ne pas retenir Kaniko, archivé depuis juin 2025 (source)
Registre de conteneurs GitLab	Images et charts au format OCI, politiques de nettoyage	Volume à surveiller ; taille maximale des paquets génériques à vérifier pour les bundles
Registres de paquets (npm, Maven, PyPI…)	Dépendances et configurations partagées	Miroir via le proxy autorisé, sinon import par le sas
Catalogue CI/CD	Composants versionnés inclus par chaque dépôt	Composants publics utiles recopiés dans l'instance
SonarQube et PostgreSQL	Analyse, quality gate, décoration des MR	L'édition Community n'analyse que la branche principale ; branches et MR à partir de Developer (source)
Politiques GitLab	Jobs obligatoires, approbations conditionnelles	Les pipelines de conformité sont dépréciés au profit des politiques d'exécution de pipeline (source)
Argo CD	Synchronise SKE depuis le dépôt GitOps	SSO GitLab, autorité de certification interne, webhook GitLab, notifications sans mail
Image Playwright	Navigateurs pour les tests d'acceptation	Image officielle importée, versions de navigateurs figées
Outils de livraison	Archive d'images (skopeo), SBOM CycloneDX, signature (cosign)	Signature par clé interne, sans journal de transparence public

Élément	Rôle	Point d'attention en enclave
Coffre de secrets	Secrets des pipelines et des déploiements	GitLab Secrets Manager (OpenBao) est en version générale depuis 19.5, avec des runners 19.0+ (source)
Banc standalone	Machine vierge réinstallée à chaque release candidate	Runner dédié équipé de docker ou podman

Environnements

Environnement	Cible	Déclenchement	Condition
dev	SKE	Automatique à chaque merge sur `main`	Aucune
intégration	SKE	Automatique pour chaque release candidate	Smoke Playwright vert
qualification	SKE	MR de promotion	Approbation du responsable de recette
production	SKE	MR de promotion, dans une fenêtre	Approbation du release manager
banc standalone	Machine vierge	Chaque release candidate	Smoke vert
sites standalone	Hors cluster	Bundle transféré par le sas	Recette du programme

Référentiels à produire

- ADR : modèle de branches, arborescence, runners, versionnement, GitOps, assemblage, standalone, secrets, promotion.
- Stratégie de test et matrice de blocage.
- Guide de l'usine pour les développeurs, procédure du sas.
- DoD commune, proposée dans Préparation du PI-1.

Rôles

Rôle	Responsabilité
Équipe usine (System Team)	Construit et maintient la chaîne, accompagne les équipes
Équipes produit	Adoptent la chaîne, écrivent TU, TI et TA
Responsable d'assemblage	Compose les versions produit et les profils programme
Release manager	Approuve les promotions en qualification et en production
Responsable qualité	Valide les seuils et le rapport de qualimétrie
Officier de sécurité	Valide le sas, les secrets, les scans et les signatures
Exploitation SKE	Fournit namespaces, flux réseau et, le cas échéant, Argo CD

Saisie dans GitLab

Les deux fonctions techniques entrent dans le modèle existant comme deux Fonctions de service, avec des Features de nature technique et des éléments au backlog de l'équipe usine.

Niveau	Type GitLab	Emplacement	Champs à renseigner
FS-T1, FS-T2	Epic, label `niveau::fonction-service`	`ldp`	Réf. exigence (exigences non fonctionnelles), Chaînes fonctionnelles servies
Features Q1 à Q7, D1 à D7	Epic, label `niveau::feature`, enfant de sa FS	`ldp`	Nature technique, jalon PI-1 ou PI-2, champs WSJF

Niveau	Type GitLab	Emplacement	Champs à renseigner
Enabler	Enabler	Backlog de l'équipe usine, par exemple `ldp/10-equipes/equipe-usine`	Catégorie enabler, poids, itération
Spike	Spike	Même backlog	Timebox (jours), livrable attendu (ADR ou note)
User Story	User Story	Backlog de l'équipe usine ou de l'équipe pilote	Critères d'acceptation, poids, itération

- Lier chaque FS technique aux FS fonctionnelles qu'elle sert (lien « lié à ») : la traçabilité besoin → solution reste visible dans `00_workitems_complet.csv`.
- Les Features marquées PI-1 → PI-2 se scindent en deux si vous appliquez la règle « une Feature tient dans un PI » ; la partie PI-2 s'affine à la préparation du PI-2.
- Les catégories d'enabler suivent SAFe (Architecture, Infrastructure, Exploration, Conformité) : à aligner sur les valeurs configurées du champ.
- Les poids s'estiment en équipe au PI planning ; les tailles S, M, L des Features ne servent qu'au séquencement.
- Ajouter à `vues.json` une vue `usine_pi1` filtrée par `descendants_de` sur les deux FS techniques, pour suivre l'avancement hors enclave.

Backlog FS-T1 — Qualimétrie intégrée à la chaîne CI/CD

FS-T1 rend la qualité mesurée, visible dans chaque MR et opposable : 7 Features, 43 éléments, dont 30 au PI-1.

Q1 · Flux de branches et protection des dépôts — PI-1 · taille S

Bénéfice attendu : `main` reste livrable, rien n'entre sans revue ni pipeline vert, l'historique alimente les notes de version.

Critères d'acceptation : 100 % des dépôts produit conformes au contrôle automatique ; push direct impossible sur `main` et `release/*`.

Réf.	Type	Élément	Critère d'acceptation	Sprint
Q1.1	Enabler · Architecture	ADR du modèle de branches : trunk-based (branches courtes depuis `main`), `release/x.y` par version livrée, hotfix et backport par MR, tags `vX.Y.Z`	ADR validé par les tech leads, schéma publié	PI1-S1
Q1.2	Enabler · Conformité	Protections standard : `main` et `release/*` protégées (merge Developer et plus, push personne), tags `v*` protégés, pipeline réussi et discussions résolues obligatoires	Appliquées au groupe et par script d'alignement (API) ; écart nul	PI1-S1
Q1.3	Enabler · Conformité	Règles de push du groupe : branche `type/numéro-sujet`, messages Conventional Commits, fichiers secrets refusés, taille maximale	Un push non conforme est refusé avec un message clair	PI1-S1
Q1.4	Enabler · Conformité	CODEOWNERS et règles d'approbation : 1 pair sur `main`, équipe usine sur `.gitlab-ci.yml`, `Dockerfile` et charts, release manager sur `release/*`	Une MR qui touche le pipeline attend l'approbation usine	PI1-S2
Q1.5	User Story	En tant que développeur, je crée branche et MR depuis l'US, avec un modèle de MR (checklist DoD) et un guide d'une page	MR liée à l'US automatiquement ; guide testé par l'équipe pilote	PI1-S2

Réf.	Type	Élément	Critère d'acceptation	Sprint
Q1.6	Enabler · Infrastructure	Pipelines de résultats fusionnés et trains de merge sur les dépôts à fort trafic	`main` ne casse plus après des merges concurrents (pilote)	PI1-S4

Q2 · Linters et rapports dans les MR — PI-1 · taille S

Bénéfice attendu : les erreurs simples sont détectées en minutes et affichées dans le diff, avec les mêmes règles pour toutes les équipes.

Critères d'acceptation : chaque stack a un linter de référence exécuté en CI ; résultats au format Code Quality dans les MR.

Réf.	Type	Élément	Critère d'acceptation	Sprint
Q2.1	Enabler · Exploration	Inventaire des stacks (langages, frameworks, versions, gestionnaires de paquets) et matrice outil × stack : linter, formatteur, TU, couverture, build	Matrice validée ; entrée de Q3, Q4 et D1.6	PI1-S1
Q2.2	Enabler · Infrastructure	Composants CI de lint par stack (front : ESLint, Prettier, Stylelint, `tsc`) et transverses (hadolint, yamllint, shellcheck, `helm lint`, kubeconform, commitlint) ; configurations partagées publiées en paquets internes	Composants au catalogue, utilisés par le pilote	PI1-S2
Q2.3	User Story	En tant que développeur, je vois les erreurs de lint dans le diff de ma MR, pour corriger avant la revue	Rapport Code Quality affiché sur la MR du pilote	PI1-S2

Réf.	Type	Élément	Critère d'acceptation	Sprint
Q2.4	User Story	En tant que développeur, j'ai les mêmes alertes dans mon IDE avant de pousser : configurations partagées, extension SonarQube (ex-SonarLint) en mode connecté	Installation hors ligne documentée, testée sur 2 postes	PI-2

Q3 · SonarQube et quality gate — PI-1 → PI-2 · taille M

Bénéfice attendu : une mesure homogène et opposable sur tout le produit, appliquée au code nouveau sans bloquer sur l'existant.

Critères d'acceptation : SonarQube en service avec connexion GitLab ; quality gate « LdP » appliqué ; tous les dépôts produit analysés en fin de PI-2.

L'édition Community n'analyse que la branche principale ; l'analyse des branches et des MR, et la décoration des MR, commencent à l'édition Developer (documentation SonarQube). Le blocage du merge passe par `sonar.qualitygate.wait=true` et le réglage « pipelines must succeed ».

Réf.	Type	Élément	Critère d'acceptation	Sprint
Q3.1	Spike · 5 j	Édition et hébergement SonarQube : Community ou Developer et plus (MR, décoration, agrégation par produit), dimensionnement, licence hors ligne, VM ou cluster outillage	Note de décision ; demande d'achat lancée si besoin	PI1-S1
Q3.2	Enabler · Infrastructure	Installation SonarQube et PostgreSQL dans l'enclave : certificats internes, sauvegarde, supervision, mises à jour par le sas	Instance en service ; une restauration réussie	PI1-S2

Réf.	Type	Élément	Critère d'acceptation	Sprint
Q3.3	Enabler · Infrastructure	Composant CI `sonar` : scanner, import couverture et résultats de tests, attente du quality gate, projet créé automatiquement (clé = chemin GitLab), jeton en variable masquée du groupe	Pilote analysé sur `main` et, selon l'édition, à chaque MR	PI1-S2
Q3.4	Enabler · Infrastructure	Connexion à SonarQube avec le compte GitLab, droits par groupe	Connexion SSO ; droits conformes aux groupes GitLab	PI1-S3
Q3.5	Enabler · Conformité	Profils qualité par langage et quality gate « LdP » sur le code nouveau ; proposition : 0 bug ou vulnérabilité bloquant ou critique, couverture ≥ 80 %, duplication ≤ 3 %, hotspots revus	Seuils validés par le responsable qualité, appliqués par défaut	PI1-S3
Q3.6	User Story	En tant que développeur, je vois dans ma MR les nouveaux problèmes et la couverture du code nouveau	Décoration active sur le pilote, ou statut du job en édition Community	PI1-S3
Q3.7	Enabler · Infrastructure	Analyse de tous les dépôts du socle et des variantes	100 % des dépôts produit dans SonarQube	PI-2

Q4 · Stratégie de test, TU et TI — PI-1 → PI-2 · taille M

Bénéfice attendu : les régressions sont détectées avant le merge et la couverture est mesurée à chaque MR.

Critères d'acceptation : stratégie validée et reprise dans la DoD ; TU à chaque MR avec résultats et couverture visibles ; TI automatisés sur le pilote.

Réf.	Type	Élément	Critère d'acceptation	Sprint
Q4.1	Enabler · Architecture	Stratégie de test de la LdP : pyramide TU, TI, TA ; ce qui tourne à chaque MR, sur `main`, la nuit, après déploiement ; données de test ; seuils ; rôles	Document validé, repris dans la DoD	PI1-S1 à S2
Q4.2	Enabler · Infrastructure	Composants CI de TU par stack avec rapports JUnit et couverture (Cobertura ou JaCoCo), transmis à SonarQube	Tests et couverture visibles dans la MR du pilote	PI1-S2
Q4.3	User Story	En tant que développeur, je vois dans ma MR les tests en échec et les lignes modifiées non couvertes	Widget de tests et couverture du diff opérationnels	PI1-S2
Q4.4	Spike · 3 j	Exécution des TI : services CI, Testcontainers (moteur de conteneurs dans le job) ou environnement éphémère sur cluster ; contraintes de sécurité des runners	Option retenue, prototypée sur le pilote	PI1-S3
Q4.5	Enabler · Infrastructure	Composant CI de TI (base, broker et jeux de données éphémères) appliqué au pilote	TI à chaque MR du pilote	PI1-S4
Q4.6	Spike · 5 j	Tests de contrat entre microservices : rupture d'API OpenAPI entre versions, option contrats consommateurs, pour sécuriser l'assemblage	Approche retenue ; une rupture d'API détectée en démo	PI-2

Réf.	Type	Élément	Critère d'acceptation	Sprint
Q4.7	Enabler · Infrastructure	Généralisation TU et TI à tous les dépôts ; tests instables en quarantaine, relance limitée	100 % des dépôts avec TU en CI ; quarantaine suivie	PI-2

Q5 · Tests d'acceptation automatisés (Playwright) — PI-1 → PI-2 · taille M

Bénéfice attendu : les critères d'acceptation sont vérifiés sur le produit assemblé ; la promotion s'appuie sur des preuves.

Critères d'acceptation : smoke exécuté automatiquement après chaque déploiement en intégration ; rapports dans GitLab ; lien entre tests et US.

Réf.	Type	Élément	Critère d'acceptation	Sprint
Q5.1	Spike · 3 j	Playwright dans l'enclave : image avec navigateurs (officielle importée ou interne), ressources et parallélisme des runners, cible (intégration ou pile éphémère), comptes de test	Smoke de démonstration exécuté dans un runner de l'enclave	PI1-S2
Q5.2	Enabler · Architecture	Dépôt `tests-acceptation` : pages, fixtures, étiquettes `@smoke`, `@regression`, `@programme-x`, configuration par environnement et par programme	Dépôt créé depuis un modèle, revu par les équipes	PI1-S3
Q5.3	Enabler · Infrastructure	Composant CI Playwright : rapport JUnit dans GitLab, rapport HTML, traces et captures en artefacts, exécution parallèle	Rapport consultable depuis le pipeline	PI1-S3

Réf.	Type	Élément	Critère d'acceptation	Sprint
Q5.4	User Story	En tant qu'équipe, je sais en quelques minutes si le produit en intégration fonctionne : le smoke se lance après chaque synchronisation Argo CD	Déclenchement automatique, résultat visible sur l'environnement	PI1-S4
Q5.5	User Story	En tant que PO, je vois quels critères d'acceptation de mes US sont couverts par un test, annoté avec la référence GitLab de l'US	Rapport de couverture des US à chaque exécution	PI-2
Q5.6	Enabler · Infrastructure	Régression nocturne sur l'intégration et sur le banc standalone, résultats historisés	Exécution planifiée chaque nuit	PI-2

Q6 · Règles de blocage et politiques de groupe — PI-1 → PI-2 · taille M

Bénéfice attendu : les règles s'appliquent à tous les dépôts sans dépendre de chaque projet ; les dérogations sont tracées.

Critères d'acceptation : matrice de blocage validée ; jobs obligatoires injectés par politique ; mode bloquant sur les pilotes en fin de PI-1, sur tous les dépôts au PI-2.

Réf.	Type	Élément	Critère d'acceptation	Sprint
Q6.1	Enabler · Conformité	Matrice de blocage : effet de chaque contrôle (lint, TU, quality gate, TI, secrets, vulnérabilités, TA) sur la MR, le merge, le tag et la promotion ; passage observation → bloquant ; dérogation tracée	Validée par qualité, sécurité et tech leads	PI1-S2

Réf.	Type	Élément	Critère d'acceptation	Sprint
Q6.2	Enabler · Conformité	Cadre de conformité « LdP » sur les dépôts produit, projet de politiques, politique d'exécution de pipeline injectant lint, sonar et détection de secrets	Un dépôt qui retire ces jobs de son `.gitlab-ci.yml` les exécute quand même	PI1-S3
Q6.3	User Story	En tant que développeur, quand mon pipeline bloque, le job indique la règle, le rapport et la marche à suivre	Messages normalisés dans tous les composants	PI1-S3
Q6.4	Enabler · Conformité	Mode bloquant (quality gate, couverture du code nouveau) sur les dépôts pilotes	Une MR qui dégrade le code nouveau ne peut plus être fusionnée	PI1-S4
Q6.5	Enabler · Conformité	Mode bloquant étendu à tous les dépôts produit, selon la trajectoire de la matrice	100 % des dépôts en mode bloquant	PI-2
Q6.6	Enabler · Conformité	Politiques d'approbation de MR : approbation sécurité si nouvelle vulnérabilité critique ou haute, ou licence non autorisée	Règle démontrée sur une MR de test	PI-2
Q6.7	User Story	En tant que responsable qualité, je reçois avec chaque version un rapport de qualimétrie (SonarQube, tests, couverture, vulnérabilités) joint à la release	Rapport généré automatiquement, exportable par le sas	PI-2

Réf.	Type	Élément	Critère d'acceptation	Sprint
Q6.8	User Story	En tant que RTE, je suis les indicateurs DORA et de flux du produit dans GitLab	Indicateurs alimentés par les jobs de promotion en environnement de niveau production (D5.4)	PI-2

Q7 · Sécurité du code dans la CI (ajout proposé) — PI-1 → PI-2 · taille M

Non demandée explicitement, cette Feature est à arbitrer : elle sert l'homologation et le dossier de livraison.

Bénéfice attendu : secrets et vulnérabilités détectés avant livraison ; registre de vulnérabilités du produit.

Critères d'acceptation : détection de secrets sur tous les dépôts au PI-1 ; SAST et analyse des dépendances au PI-2 ; bases à jour selon un circuit sas défini.

Réf.	Type	Élément	Critère d'acceptation	Sprint
Q7.1	Enabler · Conformité	Détection de secrets dans tous les pipelines, et protection au push si elle est activée sur l'instance	Un secret de test poussé est détecté ou refusé	PI1-S3
Q7.2	Spike · 5 j	Scanners hors ligne : analyseurs GitLab (SAST, dépendances, conteneurs) importés, bases d'avis et de licences, fréquence et circuit de mise à jour par le sas	Circuit validé avec l'officier de sécurité ; un scan hors ligne réussi	PI1-S4
Q7.3	Enabler · Infrastructure	SAST et analyse des dépendances sur les dépôts produit, résultats dans le rapport de vulnérabilités du groupe	100 % des dépôts produit analysés	PI-2

Réf.	Type	Élément	Critère d'acceptation	Sprint
Q7.4	Enabler · Infrastructure	Mise à jour périodique des bases de vulnérabilités par le sas, avec alerte quand une base dépasse son âge maximal	Âge des bases contrôlé automatiquement	PI-2
Q7.5	User Story	En tant qu'officier de sécurité, je consulte le registre des vulnérabilités du produit et les acceptations de risque tracées	Rapport du groupe utilisé en revue de sécurité	PI-2

Backlog FS-T2 — Chaîne de livraison de la ligne de produits

FS-T2 transforme un merge en version produit livrable sur Kubernetes et en standalone : 7 Features, 44 éléments, dont 31 au PI-1.

D1 · Socle de l'usine logicielle — PI-1 · taille L

Bénéfice attendu : une seule façon de construire, tester et publier, maintenue par l'équipe usine ; un nouveau microservice est conforme dès sa création.

Critères d'acceptation : arborescence en place ; runners, registres et miroirs opérationnels ; catalogue CI versionné ; modèles back et front ; outillage importé par le sas.

Réf.	Type	Élément	Critère d'acceptation	Sprint
D1.1	Enabler · Architecture	Arborescence GitLab (usine, produit, livraison), conventions de nommage, rôles, choix du pilote (1 back, 1 front)	ADR validé, groupes créés, pilote désigné	PI1-S1

Réf.	Type	Élément	Critère d'acceptation	Sprint
D1.2	Enabler · Infrastructure	Lot d'import par le sas : liste versionnée des images, binaires, plugins et bases (voir annexe), empreintes, circuit de validation, cycle de mise à jour	Lot 1 importé et validé ; à lancer avant le PI	PI1-S1
D1.3	Spike · 3 j	Architecture des runners : exécuteur (Kubernetes ou VM), isolation, build sans privilège (Buildah ou BuildKit rootless), moteur de conteneurs pour TI et banc standalone, flux réseau	ADR et dimensionnement	PI1-S1
D1.4	Enabler · Infrastructure	Runners de groupe étiquetés par usage (`build` , `test` , `e2e` , `standalone`), quotas, nettoyage	Pipeline du pilote exécuté sur les runners cibles	PI1-S2
D1.5	Enabler · Infrastructure	Registres et miroirs : conteneurs, npm, Maven, PyPI… via le proxy autorisé ou le sas ; gestionnaires de paquets préconfigurés dans les images de build ; politiques de nettoyage	Build du pilote sans accès externe direct	PI1-S2
D1.6	Enabler · Infrastructure	Images de base durcies par stack (build et exécution), reconstruites et rescannées par pipeline planifié	Images utilisées par le pilote ; reconstruction mensuelle	PI1-S2

Réf.	Type	Élément	Critère d'acceptation	Sprint
D1.7	Enabler · Architecture	Catalogue CI/CD : composants versionnés (SemVer), entrées documentées, tests des composants, publication au catalogue de l'instance	Le `.gitlab-ci.yml` du pilote se limite à quelques `include: component`	PI1-S2
D1.8	User Story	En tant que tech lead, je crée un microservice back ou front conforme depuis un modèle de projet : pipeline, Dockerfile, chart, configuration qualité, CODEOWNERS	Pipeline vert au premier push	PI1-S3
D1.9	User Story	En tant que développeur, je trouve dans un guide unique comment utiliser l'usine : composants, règles, déploiement, dépannage	Guide publié, revu par l'équipe pilote	PI1-S4

D2 · Build et publication des composants — PI-1 · taille M

Bénéfice attendu : chaque composant produit un artefact unique, versionné et traçable, promu sans reconstruction jusqu'en production.

Critères d'acceptation : build sans privilège standard ; versions SemVer immuables ; image et chart publiés à chaque merge et à chaque tag.

Réf.	Type	Élément	Critère d'acceptation	Sprint
D2.1	Enabler · Architecture	Convention de versionnement : SemVer par composant, version produit portée par l'assemblage, tags immuables, promotion par empreinte, génération du changelog	ADR validé, appliqué au pilote	PI1-S1
D2.2	Enabler · Infrastructure	Composant CI `build-image` : rootless, cache, étiquettes OCI (source, révision, version)	Image du pilote traçable jusqu'au commit	PI1-S2
D2.3	Enabler · Architecture	Chart Helm générique « microservice » (ou format retenu par D4.1), publié en OCI dans le registre GitLab, validé par `helm lint` et kubeconform	Pilote déployable avec ce chart	PI1-S2 à S3
D2.4	User Story	En tant que développeur, chaque merge sur `main` publie une image et un chart versionnés, traçables jusqu'au commit et à la MR	Démontré sur le pilote back et front	PI1-S3
D2.5	Enabler · Infrastructure	Migration des autres microservices du socle vers la chaîne standard, avec leurs équipes	3 composants migrés au PI-1, le reste au PI-2	PI1-S4

D3 · Assemblage de la ligne de produits et variabilité — PI-1 → PI-2 · taille L

Bénéfice attendu : une version produit est décrite une fois et déployée à l'identique sur les deux cibles ; la variabilité par programme est explicite et revue.

Critères d'acceptation : manifeste de version produit contrôlé en CI ; profils par programme ; les deux cibles générées depuis la même source.

Réf.	Type	Élément	Critère d'acceptation	Sprint
D3.1	Spike · 5 j	Modèle d'assemblage : composants et versions d'une version produit, variabilité par programme (composants inclus, paramètres, fonctions activées) ; options : chart parapluie et values par programme, Kustomize, descripteur maison et générateurs	ADR ; prototype générant les deux cibles pour le pilote	PI1-S1 à S2
D3.2	Spike · 3 j	Secrets et configuration pour les deux cibles : pipelines (GitLab Secrets Manager si version 19.5 ou plus), Kubernetes (Sealed Secrets, SOPS ou coffre existant), standalone (fichiers chiffrés, secrets du moteur)	ADR validé par l'officier de sécurité	PI1-S2
D3.3	Enabler · Architecture	Dépôt `assemblage` : manifeste de version produit, profils programme, schéma, contrôles CI (images et charts référencés existants, compatibilités)	Version produit 0.1 décrite et validée en CI	PI1-S3
D3.4	Enabler · Architecture	Migrations de schéma de données exécutées à l'identique dans les deux modes (job de migration livré avec l'image)	Migration du pilote rejouée sur Kubernetes et en standalone	PI1-S4

Réf.	Type	Élément	Critère d'acceptation	Sprint
D3.5	User Story	En tant que responsable d'assemblage, je compose la version produit d'un programme en modifiant un seul fichier, via une MR revue	Profil du premier programme livré, diff lisible	PI-2
D3.6	User Story	En tant que responsable de programme, je vois quelle version (composants et configuration) tourne sur chaque environnement et chez chaque programme	Vue consolidée, à jour sans saisie	PI-2

D4 · Déploiement Kubernetes en GitOps, dev et intégration — PI-1 · taille L

Bénéfice attendu : tout merge sur `main` arrive en dev sans intervention ; l'état des environnements est décrit dans Git, auditable et réversible.

Critères d'acceptation : Argo CD synchronise dev et intégration depuis le dépôt GitOps ; déploiement automatique du pilote ; retour arrière par revert démontré.

Réf.	Type	Élément	Critère d'acceptation	Sprint
D4.1	Spike · 5 j	Architecture GitOps : dépôts, arborescence environnements × programmes, ApplicationSets, AppProjects et RBAC, mise à jour des versions (commit par la CI ou Argo CD Image Updater), rendu Helm par Argo CD ou manifestes rendus en CI	ADR ; démonstration sur un namespace de test	PI1-S1

Réf.	Type	Élément	Critère d'acceptation	Sprint
D4.2	Enabler · Infrastructure	Prérequis SKE avec l'équipe d'exploitation : namespaces et quotas, ingress et certificats, classes de stockage, accès au registre (jeton de déploiement), flux GitLab ↔ Argo CD ↔ cluster	Namespaces prêts ; un pod tire une image du registre GitLab	PI1-S1 à S2
D4.3	Enabler · Infrastructure	Argo CD raccordé à GitLab : lecture des dépôts, autorité de certification interne, SSO GitLab et RBAC par groupe, webhook pour synchronisation immédiate, notifications par webhook	Connexion SSO ; synchronisation dans la minute après un commit	PI1-S2
D4.4	Enabler · Infrastructure	Dépôt GitOps et ApplicationSets : dev (synchronisation automatique, prune, self-heal) et intégration	Dev et intégration décrits entièrement dans Git	PI1-S3
D4.5	Enabler · Infrastructure	Composant CI `gitops-update` : commit de montée de version par un compte de service, déploiement enregistré dans l'environnement GitLab	Chaque déploiement visible dans GitLab	PI1-S3
D4.6	User Story	En tant que développeur, ma modification fusionnée sur `main` est déployée en dev sans intervention, et je vois l'état de synchronisation	Démontré sur le pilote back et front	PI1-S3

Réf.	Type	Élément	Critère d'acceptation	Sprint
D4.7	User Story	En tant qu'équipe, je reviens à la version précédente en dev ou en intégration par un revert dans le dépôt GitOps	Retour arrière démontré en démo système	PI1-S4

D5 · Promotion vers qualification et production — PI-1 (cadre) → PI-2 · taille M

Bénéfice attendu : le même artefact validé est promu jusqu'en production par des MR approuvées ; chaque mise en production est tracée et réversible.

Critères d'acceptation : règles de promotion validées ; promotion par MR avec approbations ; retour arrière en production testé.

Réf.	Type	Élément	Critère d'acceptation	Sprint
D5.1	Enabler · Architecture	Chaîne d'environnements et règles de promotion : dev (auto), intégration (auto, smoke vert), qualification (MR approuvée), production (MR du release manager, fenêtre) ; promotion par empreinte	ADR validé par l'exploitation et la qualité	PI1-S4
D5.2	Enabler · Conformité	Environnements protégés et approbations de déploiement GitLab, CODEOWNERS sur les répertoires qualification et production du dépôt GitOps, gels de déploiement	Promotion en production impossible sans approbation	PI-2

Réf.	Type	Élément	Critère d'acceptation	Sprint
D5.3	Enabler · Infrastructure	Argo CD en qualification et production : synchronisation contrôlée, fenêtres de synchronisation alignées sur les créneaux de mise en production	Synchronisation hors fenêtre refusée	PI-2
D5.4	User Story	En tant que release manager, je promeus une version validée en qualification puis en production par une MR qui montre le diff exact et exige les approbations	Première mise en production par ce circuit ; job déclarant un environnement de niveau production	PI-2
D5.5	User Story	En tant qu'exploitant, je reviens en version N-1 en production avec une procédure testée	Exercice de retour arrière chronométré	PI-2

D6 · Livraison standalone (docker, podman) — PI-1 (MVP) → PI-2 · taille L

Bénéfice attendu : le produit s'installe et se met à jour sans cluster ni accès à GitLab, depuis un paquet hors ligne contrôlé, avec les mêmes images que Kubernetes.

Critères d'acceptation : format choisi ; bundle généré par la CI depuis l'assemblage ; installation en une commande sur machine vierge ; intégrité vérifiable ; bundle testé automatiquement.

Compose fonctionne avec docker comme avec podman ; Quadlet et `podman kube play` sont propres à podman. Si les sites ont les deux moteurs, Compose est le dénominateur commun.

Réf.	Type	Élément	Critère d'acceptation	Sprint
D6.1	Spike · 5 j	Format standalone : Compose, Quadlet (podman et systemd) ou `podman kube play` (manifestes Kubernetes) ; critères : moteurs et OS des sites, rootless, démarrage au boot, SELinux, point d'entrée, persistance, mise à jour	ADR, inventaire des cibles, POC sur machine représentative	PI1-S1 à S2
D6.2	Enabler · Infrastructure	Générateur de bundle en CI : archive des images chargeable par le moteur cible, fichiers générés depuis l'assemblage, configuration par programme, scripts `install` et `verify`, MANIFEST.txt et SHA256SUMS ; publié en paquet générique GitLab	Bundle v0 du pilote produit à chaque release candidate	PI1-S3
D6.3	Enabler · Architecture	Point d'entrée standalone : reverse proxy et TLS par certificats internes, à la place de l'Ingress	Front et API joignables par les mêmes chemins qu'en Kubernetes	PI1-S3
D6.4	User Story	En tant qu'intégrateur, j'installe une version complète sur une machine vierge, sans accès au cluster ni à GitLab, en une commande avec contrôle d'intégrité	`verify` vert et smoke Playwright vert	PI1-S4

Réf.	Type	Élément	Critère d'acceptation	Sprint
D6.5	Enabler · Infrastructure	Banc standalone en CI : runner dédié qui réinstalle chaque release candidate sur machine vierge et lance le smoke	Échec du banc = release candidate refusée	PI-2
D6.6	User Story	En tant qu'intégrateur, je mets à jour une installation de N vers N+1 sans perte de données et je peux revenir en N	Mise à jour et retour démontrés	PI-2
D6.7	User Story	En tant qu'exploitant standalone, je sauvegarde et restaure les données persistantes avec une procédure fournie	Restauration testée	PI-2

D7 · Release, traçabilité et intégrité des livrables — PI-1 → PI-2 · taille M

Bénéfice attendu : chaque version livrée à un programme est identifiable, complète, intègre et signée ; on sait quelles US elle contient.

Critères d'acceptation : release produit outillée ; SBOM et scan des images ; signature vérifiée à l'installation ; dossier de livraison exportable par le sas.

Réf.	Type	Élément	Critère d'acceptation	Sprint
D7.1	Enabler · Infrastructure	Pipeline de release produit : tag de l'assemblage, gel des versions, livrables Kubernetes et standalone, notes de version agrégées depuis les MR et US liées, release GitLab avec ses fichiers	Version produit v0.1 publiée en fin de PI-1	PI1-S4

Réf.	Type	Élément	Critère d'acceptation	Sprint
D7.2	Enabler · Conformité	SBOM CycloneDX par image et par version produit, scan de vulnérabilités des images joint à la release	SBOM présente dans chaque release	PI-2
D7.3	Spike · 3 j	Signature hors ligne des images et des bundles (par exemple cosign avec clé interne, sans journal de transparence public), vérification à l'installation et côté cluster	ADR ; signature et vérification démontrées	PI-2
D7.4	User Story	En tant que responsable de recette d'un programme, je reçois un dossier de livraison cohérent : notes de version, SBOM, rapport de qualimétrie, empreintes, signature	Dossier généré automatiquement pour une release	PI-2
D7.5	Enabler · Conformité	Sortie des livrables par le sas : paquet, MANIFEST.txt, SHA256SUMS, contrôle des deux côtés, mêmes conventions que le kit d'export	Procédure validée par l'officier de sécurité	PI-2

Préparation du PI-1

Le PI-1 tient s'il démarre avec le lot d'outillage importé, l'accès au cluster SKE demandé et un pilote désigné ; le reste se séquence par prérequis.

Séquencement des Features

Ordre	Feature	PI	Taille	Prérequis
1	D1 Socle de l'usine	PI-1	L	Lot sas, flux réseau
2	Q1 Flux de branches	PI-1	S	D1.1
3	D2 Build et publication	PI-1	M	D1
4	Q2 Linters	PI-1	S	Q2.1, D1.7
5	Q3 SonarQube	PI-1 → PI-2	M	Q3.1 (édition, licence)
6	Q4 Tests TU et TI	PI-1 → PI-2	M	Q2.1, D1.7
7	D4 GitOps dev et intégration	PI-1	L	D2, accès SKE
8	D3 Assemblage	PI-1 → PI-2	L	D2
9	D6 Standalone	PI-1 → PI-2	L	D2, D3
10	Q5 Tests d'acceptation	PI-1 → PI-2	M	D4
11	Q6 Règles de blocage	PI-1 → PI-2	M	Q2, Q3, Q4
12	D7 Release	PI-1 → PI-2	M	D3, D6
13	Q7 Sécurité du code	PI-1 → PI-2	M	D1.2
14	D5 Promotion qualification et production	PI-1 → PI-2	M	D4, Q5

Pour les champs WSJF, la valeur de ces Features tient surtout à la réduction de risque et à l'opportunité ouverte ; les prérequis D1, Q1 et D2 passent en tête quel que soit le score.

Feuille de route du PI-1

Le PI-1 mène le pilote jusqu'à une v0.1 livrée sur les deux cibles

4 sprints d'un mois, par axe de travail ; un jalon en fin de sprint

	PI1-S1	PI1-S2	PI1-S3	PI1-S4
Socle et build	Arborescence GitLab lot d'import sas spike runners ADR versions	Runners, registres images de base catalogue CI build rootless	Modèles de projet publication auto image et chart	3 composants migrés guide de l'usine
Qualimétrie	ADR branches protections, push inventaire stacks spike SonarQube	Lint et TU en MR SonarQube installé matrice de blocage spike Playwright	Quality gate LdP décoration des MR politique pipeline TA : dépôt et CI	Bloquant (pilotes) TI du pilote smoke après synchro trains de merge
Kubernetes GitOps	Spike GitOps namespaces et flux demandés à SKE	Argo CD raccordé SSO, webhook	Dev et intégration en GitOps déploiement auto	Retour arrière ADR promotion
Assemblage et standalone	Spike assemblage spike standalone inventaire des sites	Spike secrets POC standalone	Manifeste v0.1 bundle v0 reverse proxy	Installation sur machine vierge release v0.1
Jalons	◆ **Architecture décidée (ADR)**	◆ **MR du pilote contrôlée**	◆ **Pilote déployé en dev sur SKE**	◆ **v0.1 livrée sur 2 cibles**

feuille de route PI-1 · 4 axes, 4 sprints, 4 jalons

Chaque jalon rapproche le pilote de la v0.1 ; un retard du lot sas ou des accès SKE décale toute la colonne suivante.

Si la capacité de l'équipe usine ne suffit pas, protéger le chemin pilote (D1, D2, D4, Q1, Q3) et passer en non engagé les éléments de PI1-S4 hors de ce chemin : Q1.6, Q7.2, D1.9, D5.1.

Objectifs PI-1 proposés pour l'équipe usine

Objectif	Engagement	Features
Le pilote (1 back, 1 front) passe du commit au déploiement en dev sur SKE par Argo CD, avec lint, TU et SonarQube visibles dans la MR	Engagé	D1, D2, D4, Q2, Q3, Q4
Les règles de flux de branches s'appliquent à tous les dépôts produit	Engagé	Q1

Objectif	Engagement	Features
La version produit v0.1 s'installe sur une machine vierge depuis un bundle hors ligne contrôlé	Engagé	D3, D6, D7
Les décisions d'architecture de l'usine sont prises et publiées (ADR)	Engagé	Spikes de PI1-S1 et S2
Le quality gate est bloquant sur les dépôts pilotes	Non engagé	Q6
Le smoke Playwright s'exécute après chaque déploiement en intégration	Non engagé	Q5
Trois microservices supplémentaires utilisent la chaîne standard	Non engagé	D2.5

La valeur métier planifiée est attribuée par les Business Owners au PI planning.

Dépendances à porter au tableau de programme

Dépendance	Fournisseur	Pour	Échéance
Lot d'outillage importé et validé	Sas, officier de sécurité	D1.2, tout le PI	Avant PI1-S1
Licence SonarQube, si édition Developer ou plus	Achats	Q3.2, Q3.6	PI1-S2
Namespaces, flux réseau, accès au registre, Argo CD	Exploitation SKE	D4.2, D4.3	PI1-S1 à S2
Équipes pilotes (1 back, 1 front) disponibles	Équipes produit	D2.4, D4.6, Q2.3	PI1-S2 à S3
Capacité réservée pour migrer leurs dépôts	Équipes produit	D2.5, Q6.5	PI1-S4 et PI-2
Machine standalone représentative, inventaire des sites	Intégration, programmes	D6.1, D6.4	PI1-S1 et S3
Validation des seuils qualité	Responsable qualité	Q3.5, Q6.1	PI1-S2
Validation des secrets, scans et signatures	Officier de sécurité	D3.2, Q7, D7.3	PI1-S2 et PI-2

Risques, avec une proposition ROAM

Risque	Impact	Réponse proposée	ROAM
Délai d'import par le sas	PI1-S2 bloqué sans outils	Lot unique préparé avant le PI, un responsable, point hebdomadaire	Atténué
Analyse des MR réservée aux éditions payantes de SonarQube	Pas de quality gate au niveau MR	Spike en S1, achat anticipé ; repli : analyse de `main` et linters bloquants	Atténué
Flux et droits sur SKE en retard	Pas de déploiement dev en S3	Demande en S1, interlocuteur de l'exploitation présent au PI planning	Pris en charge
Runners sans privilège	Build, TI ou banc impossibles	Spike runners en S1, build rootless	Atténué
Cibles standalone hétérogènes	Format inadapté à certains sites	Inventaire et POC sur machine représentative	Atténué
Capacité des équipes produit	Généralisation retardée	Capacité réservée dans leurs objectifs PI	Pris en charge
Tests Playwright instables	Perte de confiance dans les blocages	Smoke réduit, quarantaine, relance limitée	Atténué
Homologation des nouveaux outils	Mise en service retardée	Officier de sécurité associé dès S1	Pris en charge

DoD commune proposée

Appliquée aux équipes pilotes au PI-1, à toutes les équipes au PI-2.

- MR liée à une US, approuvée selon CODEOWNERS, discussions résolues.
- Pipeline vert : lint, TU, quality gate sur le code nouveau, détection de secrets.
- TU écrits pour le code nouveau, couverture au seuil.
- Image et chart publiés, déployés en dev par GitOps.
- Smoke vert en intégration ; critères d'acceptation automatisés quand ils décrivent un parcours.

- Documentation à jour, messages de commit conventionnels.

Actions avant le PI

- [ ] Lancer le lot d'import outillage par le sas (annexe).
- [ ] Qualifier le besoin d'édition SonarQube, préparer la demande d'achat.
- [ ] Demander à l'exploitation SKE les namespaces dev et intégration, les flux réseau, l'accès au registre et le statut d'Argo CD.
- [ ] Désigner le pilote (1 back, 1 front) et ses tech leads.
- [ ] Identifier une machine standalone représentative, lister les sites cibles.
- [ ] Créer dans GitLab les 2 FS techniques, les 14 Features et les éléments du PI-1.
- [ ] Inviter au PI planning l'exploitation SKE, l'officier de sécurité et le responsable qualité.

Annexe — Lot d'import outillage par le sas

Ce lot couvre le PI-1 ; l'équipe usine fige versions et empreintes avant l'import, et le rejoue à chaque mise à jour.

Élément	Usage	Feature
Images GitLab Runner et helper	Exécution des jobs	D1.4
Buildah ou BuildKit rootless, skopeo	Build et archive des images	D2.2, D6.2
Images de base par stack (JDK, Node.js, Python…)	Build et exécution	D1.6
SonarQube Server, PostgreSQL, scanner, plugins de langage	Qualimétrie	Q3
ESLint, Prettier, Stylelint, hadolint, yamllint, shellcheck, commitlint	Lint	Q2
Helm, kubeconform, schémas JSON Kubernetes et CRD	Validation des manifestes	D2.3
Argo CD et ses images, s'il n'est pas fourni	GitOps	D4.3
Image Playwright avec navigateurs	Tests d'acceptation	Q5

Élément	Usage	Feature
Analyseurs de sécurité GitLab, scanner d'images, bases de vulnérabilités	Sécurité	Q7, D7.2
Générateur SBOM CycloneDX, cosign	Release	D7
Composants CI publics utiles, recopiés dans l'instance	Catalogue	D1.7
Paquets npm, Maven, PyPI hors liste blanche du proxy	Dépendances	D1.5

Sources

- SonarQube Server 10.4 — intégration GitLab : éditions, analyse des branches et des MR
- GitLab — pipelines de conformité dépréciés, remplacés par les politiques d'exécution de pipeline
- GitLab — Secrets Manager (OpenBao) : historique des versions et prérequis
- ZenML — avertissement sur l'archivage de Kaniko en juin 2025